CLASS X ICT — EXAM REVISION NOTES

9 chapters · Python programs · flowcharts · Excel · cloud & copyright · every past-paper question solved

Built from the **BCSE 2025** and **2024** Information & Communication Technology question papers (Final I). Every Section A item and every Section B part in both papers is answered inside.

How to use: Orange tags show where a topic appeared: P25 Q7 = 2025 paper question 7; P24 Q5a = 2024 paper question 5(a). "Sec A" = Q1 (MCQ, blanks, true/false). Green boxes are **solved questions** with the mark value in brackets and the key answer in **red**. Grey boxes are **Python code** you can type and run. Every program in the notes is the kind the exam asks you to **write by hand**, so practise writing them without looking.

⚠ **Read this first — limits of these notes** I had the 2025 and 2024 papers only (no marking schemes, no textbook), so answers are the standard Class X ICT answers. Some questions in the papers are faulty or unclear; they are marked **(check)**: **(a) 2025 Q1a(xv)** — `=AVERAGE(A1:A6,">60")` is not a valid way to average values above 60 and none of the options is correct (the right function is `AVERAGEIF`, giving 75); **(b) 2025 Q5e** — the base case is printed as `text == " "` (a space); it should be the empty string `""`, otherwise the recursion would fail at the end; **(c) 2024 Q5a** — `Pass` and `Continue` are printed with capital letters; in Python they are `pass` and `continue`; **(d) 2024 Q1c(v)** and **Q6c** depend on how your textbook words the point; **(e) 2024 Q4d** — the scenario for the list-vs-set question was missing from my copy, so the answer is general. In the exam, answer the question as printed and add the correct term.

How marks are given (both papers)

- **Section A (40 marks):** Q1(a) = **15 MCQs worth 2 marks each (30)**, Q1(b) 5 blanks (5), Q1(c) 5 true/false (5). Circle only **ONE** letter.
- **Section B (60 marks):** attempt **ALL** of Q2–Q7, **10 marks each**. Q2–Q4 are mostly short theory (cloud, citation, Excel, data structures); Q5–Q7 are **programming**; the last question (Q7) is always a long program.
- **Code answers:** write correct **indentation** and colons; one mark per correct step (variable, loop, condition, print). Use meaningful names; comments are optional but help.
- **"Explain / justify / analyse":** give the point *and* the reason ("because…"). **"Write the output":** write exactly what is printed, one value per line unless `end=" "`.

Paper pattern (2024 & 2025)

Part	Content	Marks
Q1(a)	15 MCQ (code reading, Excel, cloud, copyright, APA)	30
Q1(b)	Fill in the blanks	5
Q1(c)	True / False	5
Sec B Q2	Cloud, Google Workspace, Excel (graph or data table)	10
Q3	Digital citizenship + loops / data types	10
Q4	Strings, lists, tuples, sets, dictionaries (2024: licences, charts, Turtle, sets)	10
Q5	Functions, parameters, recursion (2024: loops, while, dictionary)	10
Q6	Conditions, loops, flowchart to code, Boolean logic	10
Q7	Long program (2025: mobile banking; 2024: school mess system)	10

Topic map: where each chapter appeared

Chapter	2024	2025
1 Cloud & Google Workspace	Q1a(ii,viii),c(i),Q2b,Q3a	Q1a(iii,vi),b(i),c(iii),Q2b,c,d
2 Copyright, licences, APA	Q1a(v,x),b(ii,iii),c(v),Q3b,Q4a	Q1a(i,iv,xiii),b(iii),c(iv),Q3a,b,c
3 Spreadsheets	Q1a(i,iii,vi,xv),Q2a,Q4b	Q1a(ix,x,xv),b(v),c(ii),Q2a
4 Python basics	Q1a(xii,xiv),Q2c	Q1a(xi,xiv),c(i),Q3f,Q5a,Q6a,d
5 Loops & Turtle	Q1a(iv,vii,ix),c(iv),Q3c,Q4c,Q5a,b	Q1a(ii),b(iv),Q3e,Q6b,c
6 Strings & data structures	Q1a(xi,xiii),b(i,iv),c(ii),Q4d,Q5c	Q1a(v,viii),Q4a–g
7 Functions & recursion	Q1a(xiv),b(v),Q2c,Q6c	Q1a(vii),b(ii),Q5b–e
8 Algorithms & flowcharts	Q6a	Q1a(xii),c(v),Q3d,Q6e
9 Long programs	Q7	Q7

Repeated every year: cloud service models & benefits • Google Docs/Slides collaboration • copyright, Creative Commons, citing sources • Excel formulas (SUMIF, VLOOKUP, MAX, AVERAGE) • for/while loops with break/continue/pass • list/tuple/set/dictionary • function with parameters and return • flowchart → Python • one long menu/validation program.

1 · CLOUD COMPUTING & GOOGLE WORKSPACE

P25 Q1a(iii,vi),b(i),c(iii),Q2b,c,d · P24 Q1a(ii,viii),c(i),Q2b,Q3a

Key ideas

- **Cloud computing** = using servers, storage and software **over the internet** instead of on your own computer; you pay for what you use. **Cloud services provide services over the internet → TRUE** P25 Q1c(iii).
- **Cloud storage** (e.g. Google Drive) keeps files on **remote servers** instead of a local device P25 Q1b(i): servers. You can open them from any device with internet.

Benefits of cloud storage

- **Lower upfront cost:** no need to buy servers and hard disks; pay a small monthly fee (pay-as-you-go) P24 Q2b(i).
- **Scalability:** add or remove storage when the business grows or in busy seasons, instantly P24 Q2b(ii).
- **Backup and recovery:** copies are kept in several data centres, so data can be restored after hardware failure, fire or theft P24 Q2b(iii).
- **Access and sharing:** work from anywhere, share links, edit together; **automatic updates** and **reduced maintenance**.
- **Risks:** needs internet, security/privacy concerns, monthly cost, depends on the provider.

Cloud service models

Who manages what? (cloud service models)

	On-premises	IaaS	PaaS	SaaS
Applications	you manage	you manage	you manage	provider
Data	you manage	you manage	you manage	provider
Runtime / middleware	you manage	you manage	provider	provider
Operating system	you manage	you manage	provider	provider
Servers, storage, network	you manage	provider	provider	provider

Examples: IaaS = virtual servers; PaaS = app platform; SaaS = Google Docs, Gmail.
SaaS = least work for the user; IaaS = most control; PaaS = developers manage only the app and data.

Who manages what in IaaS, PaaS and SaaS (shaded orange = you manage).

Model	You get	Example / user
IaaS Infrastructure as a Service	virtual servers, storage, networks (you install the OS and apps)	Amazon EC2; IT team needing full control
PaaS Platform as a Service	a ready platform to **develop and run applications**; provider handles hardware, OS and updates	Google App Engine; **software developers** P24 Q1a(ii): D
SaaS Software as a Service	finished software used through a browser	Google Docs, Gmail, Zoom; **ordinary users**

Types of cloud

Types of cloud (deployment models)

PRIVATE
one organisation only; most control and security
Example: school or bank data

PUBLIC
shared; provider owns it; cheap, pay as you go
Example: Google Drive, Dropbox

HYBRID
private + public mixed; sensitive data kept private
Example: secure records + public website

COMMUNITY
shared by several groups with the same needs
Example: several colleges / hospitals

A school that wants high control and security of its own data should choose a PRIVATE cloud (P25 Q1a(vi)).

Private, public, hybrid and community cloud.

- A school that wants to **control its data and have high security** chooses a **private cloud** P25 Q1a(vi): A.

Google Workspace tools

Tool	Use
Google Docs	word processing; **many people edit the same document at the same time** with live updates, comments, suggestions and version history P25 Q1a(iii): B P24 Q1c(i): TRUE
Google Slides	presentations; share the deck and give each student different slides to work on at the same time P24 Q1a(viii): D
Google Sheets	spreadsheets with formulas and charts
Google Forms	surveys and quizzes; responses collected automatically
Google Drive	cloud storage to save, organise and share files
Google Meet	video meetings (not for editing a document)
Google Sites	building simple websites

Solved questions

P25 Q2b [2] Two reasons to advise cloud storage: (1) **no costly servers** to buy and maintain — the provider does it and the company pays only for what it uses; (2) data is **backed up and accessible from anywhere**, so staff can work remotely and data is safe if a computer is damaged.

P25 Q2c [2] Choose **SaaS** (or **IaaS** to remove servers): for **increased efficiency and reduced on-site server maintenance** a school IT team should use **SaaS** because the provider manages the servers, software and updates and the school just uses the applications. (IaaS removes the hardware but the team still has to manage the OS and software, so it does not reduce work as much.)

P25 Q2d [2] Two features of Google Forms: (1) **different question types** (multiple choice, short answer, scale) so the feedback is organised and easy to answer; (2) **automatic collection of responses into a Google Sheet with summary charts**, so data is gathered and analysed without manual typing. (Also: sharing by link, required questions, response validation.)

P24 Q2b [4] (i) The business **does not need to buy servers, disks or build a server room**; it rents storage and pays monthly. (ii) **Scalability**: storage can be increased or decreased quickly as inventory grows or in seasonal periods, paying only for what is used. (iii) **Backups are stored off-site in several data centres**, so after hardware failure, fire or theft the data can be **restored quickly** and business continues.

P24 Q3a [3] (i) Create a **Google Doc**, click **Share**, add classmates' emails with **Editor** access; everyone edits at the same time, uses **comments and suggestions** and the **version history** records changes. (ii) **Google Drive** is the **cloud storage** to store, organise, share and sync files from any device. (iii) Slides: **real-time collaboration**, ready templates, themes, animations and transitions; can be presented or shared online (any two).

P24 Q1a(ii) / P25 Q1a(vi) P24: a company that wants to **develop and run applications without managing hardware or software updates → PaaS (D)**. P25: a school wanting control and high security → **private cloud (A)**.

2 · DIGITAL CITIZENSHIP: COPYRIGHT, LICENCES & CITATION

P25 Q1a(i,iv,xiii),b(iii),c(iv),Q3a,b,c · P24 Q1a(v,x),b(ii,iii),c(v),Q3b,Q4a

Key ideas

- **Copyright** = the **exclusive legal right** given to the creator to print, publish, perform, film or record a work and to authorise others to do so P24 Q1b(iii): copyright. It applies automatically to books, music, photos, software and websites.
- **Plagiarism** = using someone else's work or ideas as your own. Acknowledging the source by **citing** it avoids plagiarism P24 Q1b(ii): plagiarism.
- **Using copyrighted work legally:** get **permission/consent** from the owner, use work with a suitable **licence** (e.g. Creative Commons), use it under **fair use/educational use** with credit, or use public-domain material. So a physics project may use a journal's work **with the author's consent → TRUE** P25 Q1c(iv).
- To use an online **song in a video** follow the rules by **making sure the use matches the licence** (credit alone is not enough) → **D** P25 Q1a(i).

Creative Commons (CC) licences

Creative Commons licences: from most open to most restricted

CC0	no rights reserved (public domain)
CC BY	use, share, change, even for money, but give credit
CC BY-SA	like BY, but share your changes under the same licence
CC BY-NC	remix and share, NOT for commercial use (credit needed)
CC BY-ND	share only, NO changes (credit needed)
CC BY-NC-SA	non-commercial; changes must use the same licence
CC BY-NC-ND	most restrictive: share only, no changes, non-commercial

The CC licences from most open to most restricted.

- **Creative Commons** licences let others **use, share and adapt** a work **under conditions chosen by the creator** P24 Q1a(x): B. The creator **keeps the copyright**; CC does **not** transfer ownership to the public domain, so "CC lets creators waive their copyright completely" is **FALSE** P24 Q1c(v) **(check)** (only CC0 waives rights).
- **BY** = give credit (attribution). **NC** = non-commercial only. **SA** = share alike. **ND** = no derivatives.
- **CC BY-NC:** others can **remix/share the song but not for profit** P25 Q1a(xiii): B.

Copyright vs Creative Commons

	Copyright	Creative Commons
Rule	**All rights reserved**; you need permission for every use	**Some rights reserved**; permission is given in advance with conditions
Effect	protects the creator's income and control, but limits sharing	encourages **sharing, remixing and education** while still crediting the creator

Citing sources (APA 7th edition)

- **In-text citation:** placed **inside the text**, e.g. (Dorji, 2020) P25 Q1b(iii): in-text. **Reference list:** full details at the end.
- **Book format: Author, A. A. (Year). *Title in sentence case, in italics*. Publisher.** Only the first word and proper names are capitalised; **no city** in the most recent (7th) edition.

```
Dorji, T. (2020). Public health challenges in Bhutan. Bhutan Health
Trust Fund.
        (title in italics)        -> P25 Q1a(iv): C
Smith, J. (2021). The digital age: Psychological impacts of social
media.
        HarperCollins Publishers. -> P24 Q1a(v): B (no place of
publication)
```

Ethical online behaviour

- **Cyberbullying / fake accounts** (Pema's fake profile): can lead to **school discipline** (warning, suspension), **legal action** under the cyber/ICT laws, **deleting the account**, and harm to the victim P25 Q3a.
- Good digital citizens: respect privacy, think before posting, give credit, report abuse, use strong passwords, and do not copy files without permission.

Solved questions

P25 Q3a [1] Consequences for Pema: the **school can punish him** (warning, suspension or counselling) and **the account can be reported and removed; legal action** may follow because impersonation and harassment are against the law; his classmate suffers emotional harm.

P25 Q3b [1] Proper citation shows **honesty and respect for the creator's work**, avoids plagiarism and lets readers check the source.

P25 Q3c [2] To use a picture legally and ethically: (1) **check its licence** (use images marked Creative Commons or public domain, or ask permission); (2) **follow the licence conditions and give credit** (author, title, source link) in the project.

P24 Q3b [3] (i) Reasons to cite images: **gives credit to the creator** and respects **copyright**; **avoids plagiarism** and lets others **verify the source**. (ii) Ethical points: respect **copyright and licences**, get **permission**, ensure the content is **accurate and appropriate**, and give **credit**. (iii) The team should **use Creative Commons or royalty-free images (or their own) and cite every source** in a reference slide, explaining to the teammate why this keeps the presentation honest and educational.

P24 Q4a [2] Copyright gives the creator **all rights reserved**: others must **ask permission**. Creative Commons lets the creator **allow certain uses in advance** (share, adapt) **with conditions** such as credit or non-commercial. Copyright **restricts** distribution; CC **promotes sharing** of creative works while protecting the creator's credit.

Quick: P25 i **D**; iv **C**; xiii **B**; blank iii **in-text**; c(iv) **TRUE**. P24 v **B**; x **B**; blanks ii **plagiarism**, iii **copyright**; c(v) **FALSE** (check).

3 · SPREADSHEETS (MS EXCEL / GOOGLE SHEETS)

P25 Q1a(ix,x,xv),b(v),c(ii),Q2a · P24 Q1a(i,iii,vi,xv),Q2a,Q4b

Key ideas

- A formula starts with **=**. **Cell reference** = column letter + row number (B2). **Range** = A1:A6 (from A1 to A6). Formulas **update automatically** when the data changes.
- **Relative** reference (B2) changes when copied; **absolute** reference (B2) stays fixed.

Common Excel functions

SUM / AVERAGE / MAX / MIN / COUNT	`=MAX(E2:E5), =AVERAGE(B2:E2)`
IF	`=IF(B2>=50,"Pass","Fail")`
AND / OR / NOT	`=NOT(A1>5) reverses TRUE/FALSE`
SUMIF	`=SUMIF(B2:B6,"IT",C2:C6) range, criteria, sum range`
COUNTIF / AVERAGEIF	`=COUNTIF(A1:A6,">60")`
VLOOKUP	`=VLOOKUP(value, table, col, FALSE)`
RANK	`=RANK(B2,B2:B10) position in a list (not alphabetical)`

The Excel functions asked in the papers.

Functions you must know

Function	Use / example
SUM, AVERAGE, MAX, MIN, COUNT	=SUM(B2:B6); **highest ICT score** → `=MAX(E2:E5)`; **average of all subjects for Mr Pema (row 2)** → `=AVERAGE(B2:E2)` P24 Q2a
IF	=IF(B2>=50,"Pass","Fail")
AND, OR, NOT	**NOT reverses** TRUE↔FALSE P24 Q1a(vi): B P25 Q1b(v): NOT
SUMIF	=SUMIF(*range to test*, *criteria*, *sum range*). Total salary of IT: `=SUMIF(B2:B6,"IT",C2:C6)` P24 Q1a(xv): D (department is column B, salary column C)
COUNTIF / AVERAGEIF	=COUNTIF(A1:A6,">60"); =AVERAGEIF(A1:A6,">60")
VLOOKUP	look up a value in the **first column** of a table and return a value from another column
RANK	gives the **position** of a number in a list (1st, 2nd…); it does **not** sort alphabetically → "RANK arranges data alphabetically" is **FALSE** P25 Q1c(ii)

VLOOKUP

VLOOKUP anatomy: =VLOOKUP(lookup_value, table_array, col_index, FALSE)

	A	B	C
1	**Student ID**	**Name**	**ICT Marks**
2	101	Karma Wangmo	85
3	102	Dorji Wangchuk	90
4	103	Tashi Dendup	78
5	104	Kelzang Lethro	88

table_array B2:C5 (lookup column B first)

=VLOOKUP("Tashi Dendup", B2:C5, 2, FALSE)
result: 78

1) value to find
(first column of the table)
2) table range, starting at that column
3) column number inside the table
4) FALSE = exact match

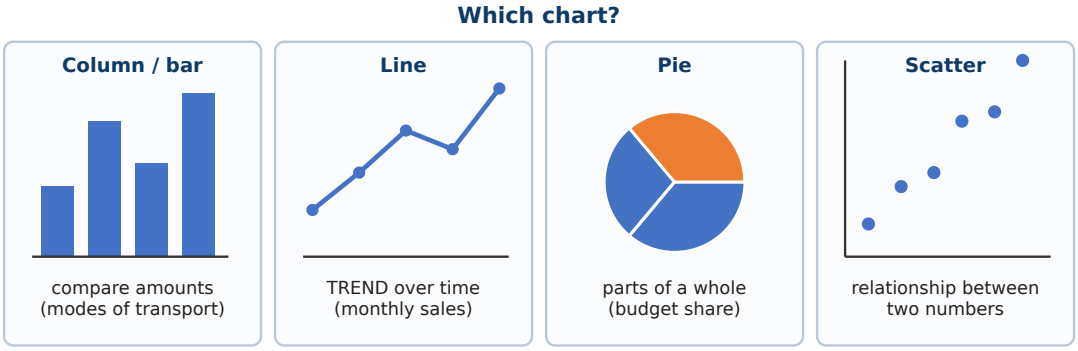
VLOOKUP: the lookup value must be in the first column of the table range.

- Tashi Dendup's score: the names are in column **B** and the marks in **C**, so the table is **B2:C5** and the marks are in column **2** of that table → `=VLOOKUP("Tashi Dendup", B2:C5, 2, FALSE)` = **78** → **A** P25 Q1a(x). The lookup value must be in the **first column** of the table range, so the range must start at column B, not A.

Other features

- **Data Validation:** restricts what can be typed, e.g. **whole numbers between 1 and 100** for test scores → **C** P24 Q1a(i).
- **Conditional formatting** colours cells by a rule; **Filter** shows rows that match a condition; **Pivot table** summarises data.
- **Total sales** = units sold × unit price. In the table (C = Units Sold, D = Unit Price) the formula in E2 is `=C2*D2` → **B** P24 Q1a(iii).

Choosing a chart



Sales trend over five months → LINE chart (P25 Q1a(ix): D)

Column, line, pie and scatter charts and when to use them.

- **Sales trend over five months → line chart** (option D) because the line joins the points in time order and shows the rise and fall P25 Q1a(ix). A pie chart shows parts of a whole; a column chart compares categories; a scatter plot shows the relationship between two numbers.
- To make a chart: select the data → **Insert → Chart** → choose the type → add title and axis labels P24 Q4b.

Reading a column graph (P25 Q2a)

Mode	Cycle	Car	Taxi	Walking	School bus
Students	4	8	6	12	10

Solved questions

P25 Q2a [4] (i) **Walking** (the tallest bar, 12). (ii) The **height of each bar shows the number of students** using that mode of transport. (iii) Students not taking the bus = total − bus users = 40 − 10 = **30 students** (4 + 8 + 6 + 12 = 30).

P24 Q2a [2] (i) **=MAX(E2:E5)** (ICT marks are in column E). (ii) **=AVERAGE(B2:E2)** (Pema's marks are in row 2, columns B to E).

P24 Q4b [2] Mr Dorji should use the **chart (Insert → Chart; a line chart)** feature. It turns the figures into a **picture**, so **trends and patterns** such as rising or falling months and the best/worst month are seen at once, and comparisons are easier than reading numbers.

P25 Q1a(xv) — (check) =AVERAGE(A1:A6,">60") is not a valid conditional average: a text argument is not a criteria test (Excel gives **#VALUE!**), and the plain average of 45, 70, 55, 80, 40 is **58**. None of the options is correct. The right formula is **=AVERAGEIF(A1:A6,">60")** = (70 + 80) ÷ 2 = **75**. Ask your teacher; in the exam circle the option your key says.

Quick: P25 ix **D** (line); x **A**; b(v) **NOT**; c(ii) **FALSE**. P24 i **C**; iii **B**; vi **B**; xv **D**.

4 · PYTHON BASICS: VARIABLES, DATA TYPES, OPERATORS & DECISIONS

P25 Q1a(xi,xiv),c(i),Q3f,Q5a,Q6a,d · P24 Q1a(xii,xiv),Q2c,Q6b

Variables & data types

- A **variable** stores a value: `age = 15`. Python is **case sensitive**: `Fruit` and `fruit` are **different** variables → **TRUE** P25 Q1c(i); `"Apple" == "apple"` is **False** P25 Q1a(xi): C.
- **Valid variable names:** letters, digits and underscore; **cannot start with a digit; no spaces or hyphens**; cannot be a keyword. `school_1` is valid; `school-1`, `school 1` and `1school` are not → **B** P24 Q1a(xii).

Type	Meaning	Example
int	whole number	books = 120 — **number of books** P25 Q1a(xiv): D
float	decimal number	price = 7.5
str	text in quotes	name = "Karma"
bool	True or False	passed = True

- **Election simulation** P25 Q3f**:** candidate name → **str** (text); number of votes → **int** (a whole number you can add and compare).
- Convert input: `n = int(input("Enter: "))` (`input()` always gives a string). Use `float()` for decimals.

Operators

Kind	Operators
Arithmetic	`+ - * /` (divide), `//` (floor), `%` (remainder), `**` (power)
Comparison	`== != < > <= >=` → give True/False
Logical	`and`, `or`, `not`
Assignment	`= += -= *=`

Boolean logic table (P24 Q6b)

X	Y	NOT Y	X AND Y	X OR Y
True	4 >= 7 = **False**	**True**	**False**	**True**
False	5 != 9 = **True**	**False**	**False**	**True**

- **AND** is True only if both are True; **OR** is True if at least one is True; **NOT** flips the value.

Decisions (if / elif / else)

```
num_1 = 7
if num_1 > 0:
    print("positive")
elif num_1 == 0:
    print("zero")
else:
    print("negative")
```

- Indentation (4 spaces) shows which lines belong to the block; each condition line ends with a colon.
- **Team A** `num_1 > 0` **vs Team B** `num_1 >= 1` P25 Q6a**:** a positive number is any number **greater than zero**, including decimals like 0.5 and 0.01. `>= 1` wrongly rejects these, so **Team A is more appropriate** (it works for all real numbers and says exactly what "positive" means).

Solved questions

P25 Q3f [2] Name: **string (str)** because a name is text. Votes: **integer (int)** because votes are counted in whole numbers and must be added and compared.

P25 Q6d [2] Circumference of a circle with r = 20:

```
r = 20
c = 2 * 3.14 * r
print("Circumference =", c)    # 125.6
```

P25 Q5a [1] Store the price returned by the function:

```
laptop_cost = get_price("laptop")
```

P24 Q1a(xiv) [2] Profit = selling price − cost price, so the function that **returns sp − cp** is correct (option D): `profit = sp - cp; calculate_profit(50, 70)` gives **20**.

P24 Q6b [3] See the Boolean table above: row 1 → False, True, False, True; row 2 → True, False, False, True (Y, NOT Y, X AND Y, X OR Y) — for row 1 Y = False (4 >= 7), NOT Y = True, X AND Y = False, X OR Y = True; row 2 Y = True (5 != 9), NOT Y = False, X AND Y = False, X OR Y = True.

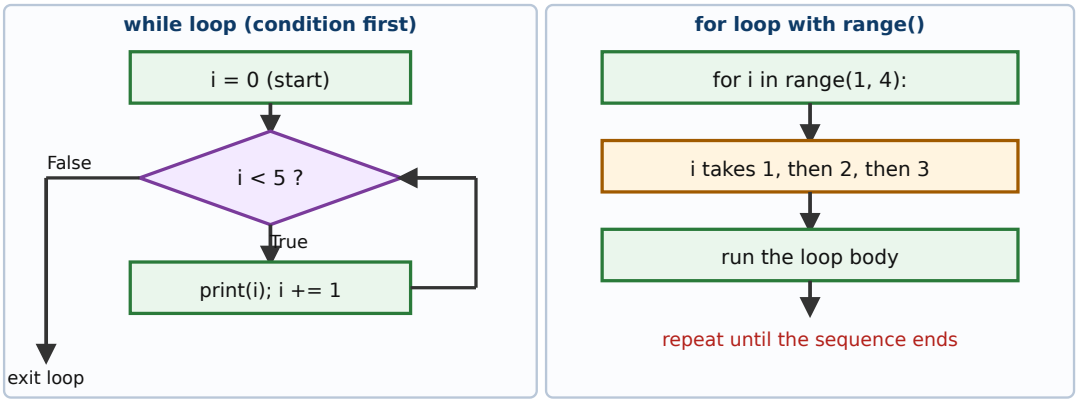
Quick: P25 xi **C**; xiv **D**; c(i) **TRUE**. P24 xii **B**; xiv **D**.

5 · LOOPS & PYTHON TURTLE

P25 Q1a(ii),b(iv),Q3e,Q6b,c · P24 Q1a(iv,vii,ix),c(iv),Q3c,Q4c,Q5a,b

for loop and range()

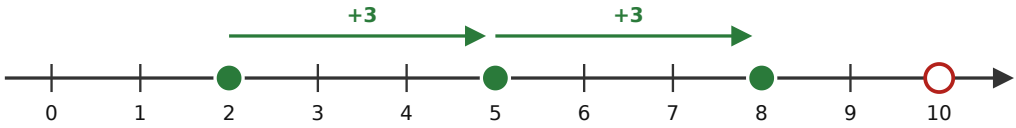
How loops work



How a while loop and a for loop run.

- Use a **for loop** when you know **how many times** to repeat P24 Q1a(vii): D; use a **while loop** when you repeat **as long as a condition is true** P24 Q1c(iv): TRUE.
- `range(stop)` = 0 to stop−1; `range(start, stop)`; `range(start, stop, step)`. **The stop value is never included.**

range(start, stop, step): stop is NOT included



stop = 10 (not printed)

range(2, 10, 3) → 2 5 8 range(1, 4) → 1 2 3 range(3, 0, -1) → 3 2 1 range(5) → 0 1 2 3 4

range() on a number line.

```
for i in range(1, 4):      # 1 2 3
    print(i)
for j in range(3, 0, -1):  # 3 2 1
    print(j)
for i in range(2, 10, 3):  # 2 5 8
    print(i, end=" ")      # end=" " keeps output on one line
```

- Output of `for i in range(1,4)` followed by `for j in range(3,0,-1)` P25 Q3e: **1 2 3 3 2 1** (each on its own line, six lines). `range(2,10,3)` with `end=" "` prints **2 5 8** P25 Q6c.
- **Even numbers 1 to 15:** `for i in range(1,15): if i % 2 == 0: print(i)` prints 2, 4, 6, 8, 10, 12, 14 → **A** P24 Q1a(iv) (option C misses 14; D prints 0).

break, continue, pass

Keyword	Effect
break	**stops the whole loop** immediately
continue	**skips the rest of this iteration** and moves to the next one P25 Q1b(iv): continue
pass	does nothing; a placeholder where a statement is needed

```
for i in range(8):
    if i == 1:
        pass          # does nothing
    elif i == 3:
        continue      # skip 3
    elif i == 5:
        break         # stop the loop
    else:
        print(i)      # prints 0, 2, 4
```

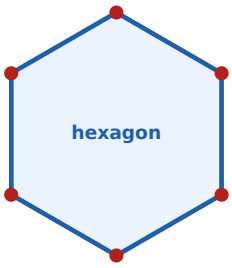
while loop

```
i = 0
while i < 5:
    if i == 3:
        break
    i += 1
    print(i)      # prints 1 2 3
```

- Trace P25 Q1a(ii): i = 0 → i becomes 1, print 1; → 2, print 2; → 3, print 3; now i == 3 so **break** before printing 4. Output **1, 2, 3** → **C**.
- Remember to **update the variable** (`i += 1`); otherwise the loop never stops (infinite loop).

Python Turtle graphics

Turtle: a regular polygon turns 360° / sides




```
import turtle
t = turtle.Turtle()
for _ in range(6):
    t.forward(100)
    t.right(60)
turtle.done()
```

6 sides, turns of 60° → HEXAGON (P24 Q1a(ix): D)

Square: 4 sides, turn 90° (360/4). Triangle: turn 120°. Pentagon: 72°. Octagon: 45°.

The turtle draws a regular polygon: turn = 360 ÷ number of sides.

```
import turtle
t = turtle.Turtle()

def draw_square():
    for _ in range(4):
        t.forward(100)
        t.right(90)

draw_square()       # calling the function runs the drawing
turtle.done()
```

- Six sides with 60° turns draw a **hexagon** P24 Q1a(ix): D.

Solved questions

P25 Q3e [2] Output:

```
1
2
3
3
2
1
```

P25 Q6b [2] The **for loop** is more efficient for printing 1 to 5: (1) the number of repetitions is **known**, and `range(1,6)` controls the counter automatically; (2) it needs **fewer lines** and there is **no risk of forgetting to increment** (an infinite loop) as in a while loop.

```
for i in range(1, 6):      i = 1
    print(i)               while i <= 5:
                               print(i)
                               i += 1
```

P24 Q3c [4] (i)

```
total = 0
for i in range(1, 51):
    total += i
print("Sum =", total)     # 1275
```

(ii)

```
i = 0
while i <= 10:
    print(i)
    i += 1
```

P24 Q4c [3] (i)–(ii) Code above: `def draw_square():` with `for _ in range(4): t.forward(100); t.right(90)`. (iii) Call it by writing the function name with brackets: **`draw_square()`**.

P24 Q5a [3] (i) Output **0, 2, 4** (each on a new line): i = 0 prints 0; i = 1 → pass; i = 2 prints 2; i = 3 → continue; i = 4 prints 4; i = 5 → break. (ii) **pass** does nothing and the loop carries on (placeholder); **break** ends the loop when i = 5, so 6 and 7 are never reached. **(check)** the paper prints `Pass` / `Continue` with capitals — the correct Python keywords are lowercase.

P24 Q5b [5] (i) `while num <= 5:` (ii) `num += 1` (or `num = num + 1`). Result of the product loop: 1×2×3×4×5 = 120. (iii) Sum 1 to 10:

```
total = 0
num = 1
while num <= 10:
    total += num
    num += 1
print("The sum of the first 10 numbers is:", total)   # 55
```

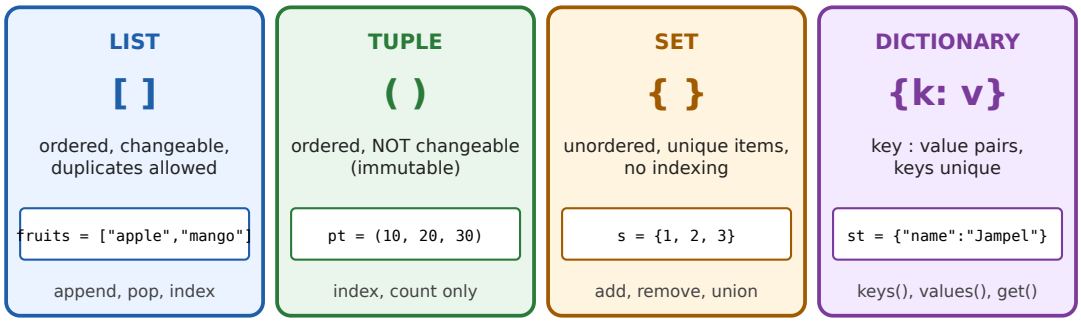
Quick: P25 ii **C**; b(iv) **continue**. P24 iv **A**; vii **D**; ix **D**; c(iv) **TRUE**.

6 · STRINGS & DATA STRUCTURES: LIST, TUPLE, SET, DICTIONARY

P25 Q1a(v,viii),Q4a–g · P24 Q1a(xi,xiii),b(i,iv),c(ii),Q4d,Q5c

Overview

Python data structures at a glance



The four data structures: brackets, order, changeability.

	List	Tuple	Set	Dictionary
Brackets	[ ]	( )	{ }	{key: value}
Ordered?	yes	yes	no	yes (3.7+)
Changeable?	**yes**	**no** (immutable)	yes	yes
Duplicates	allowed	allowed	**not allowed**	keys unique

- An **ordered, changeable collection that allows duplicates** is a **list** P24 Q1b(i). The structure that stores **key-value pairs with unique keys** is a **dictionary** P24 Q1b(iv). **Lists change; tuples stay fixed → C** P25 Q1a(v).
- To keep a collection with **no duplicates where order does not matter** use a **set** P24 Q1c(ii): TRUE.

Strings

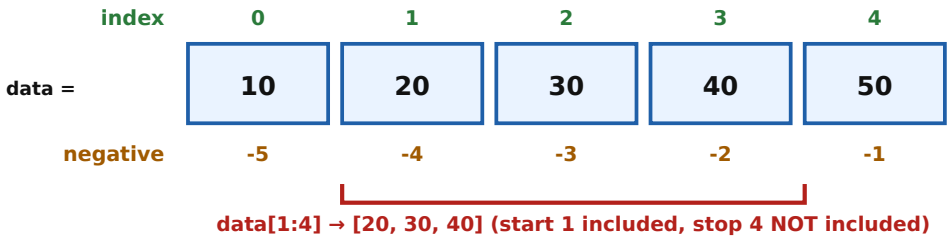
- A string is a sequence of characters; it is **immutable**: **once created, its characters cannot be changed in place** P25 Q4a. To "change" it you create a **new string**.

```
car = "Alto"
car[0] = "B"          # TypeError: strings are immutable
car = "B" + car[1:]   # new string "Balto" (the old one is replaced)
print(car)
```

- **P25 Q4b:** modifying `car` shows immutability — direct item assignment fails, so a new string must be made (e.g. `car = car + "800"` or `"B" + car[1:]`).

Lists: index, slice, methods

Indexing and slicing a list



data[0:3] → [10,20,30] data[1:3] → [20,30] data[2:5] → [30,40,50] data[-1] → 50 data[:2] → [10,20]

colors = ["red","green","blue"] → colors[-1] is "blue" (last item) (P25 Q4c)

Positive and negative indexes, and slicing.

```
data = [10, 20, 30, 40, 50]
print(data[1:4])      # [20, 30, 40]  -> C
colors = ["red", "green", "blue"]
print(colors[-1])     # blue (last item)
my_list = ["Range", "function", "inbuilt", "variable"]
my_list.pop(2)        # removes index 2 ("inbuilt")
print(my_list)        # ['Range', 'function', 'variable']
```

- Slice `data[1:4]` gives **[20, 30, 40]** (start included, stop not) → **C** P25 Q1a(viii). `colors[-1]` counts from the end, so the output is **blue** P25 Q4c. After `pop(2)` the list is **['Range', 'function', 'variable'] → D** P24 Q1a(xi).
- Methods: `append(x)`, `insert(i,x)`, `remove(x)`, `pop(i)`, `sort()`, `len(list)`.

Tuples

```
my_tuple = (10, 20, 30, 40)
print(my_tuple[1])    # 20 -> B
```

- Index starts at 0, so `my_tuple[1]` is **20** P24 Q1a(xiii). Tuples use **( )** and cannot be changed; dictionaries use **{ }** with key:value pairs and can be changed P25 Q4f.

Sets

```
attended  = {"Rina", "Karma", "Sonam", "Tashi"}
submitted = {"Rina", "Sonam"}
not_submitted = attended - submitted      # or
attended.difference(submitted)
print(not_submitted)                      # {'Karma', 'Tashi'}
```

- Set operations: union `|`, intersection `&`, difference `-`. A set **has no duplicates and no order** P24 Q4d.

Dictionaries

```
student = {"name": "Jampel", "age": 20}
student["grade"] = "A"          # add a new key
print(student)
vehicle = {'brand': 'Toyota', 'model': 'VigoHilux', 'year': 2021}
print(vehicle['model'])         # VigoHilux  (or vehicle.get('model'))
```

- `vehicle` is a **dictionary** (keys and values) P24 Q5c(i); read a value with its key: `vehicle['model']` P24 Q5c(ii).
- **Student names with scores:** a **dictionary is better than a list**: each name (key) is linked to a score (value) and the score is found instantly by name, while a list would need index positions or two parallel lists P25 Q4e.

Solved questions

P25 Q4a,b [3] Immutability: **a string cannot be changed after it is created** (you cannot replace one letter). Example: `car = "Alto"; car[0] = "B"` gives a **TypeError**; the only way is to build a new string, `car = "B" + car[1:]`.

P25 Q4d [2] Code A − code B using sets: `print(attended - submitted)` (or `attended.difference(submitted)`). The names displayed are **Karma** and **Tashi**.

P25 Q4e,f [2] (e) **Yes**, a dictionary is better: names are keys and scores are values, so a score is found directly by name. (f) A **tuple** is an ordered collection in **( )** that **cannot be changed**; a **dictionary** stores **key:value pairs** in **{ }** and **can be changed**.

P25 Q4g [2]

```
student = {"name": "Jampel", "age": 20}
student["grade"] = "A"
print(student)     # {'name': 'Jampel', 'age': 20, 'grade': 'A'}
```

P24 Q4d [3] (i) A **set** is an **unordered collection of unique items** written in { }. (ii) **(check — scenario not in my copy)** If the data must have **no repeated items** and the order does not matter (e.g. student IDs or names who attended), recommend a **set**: (1) it **removes duplicates automatically**; (2) membership checks and operations (union, difference) are **fast and simple**. If order or duplicates matter, a list is better.

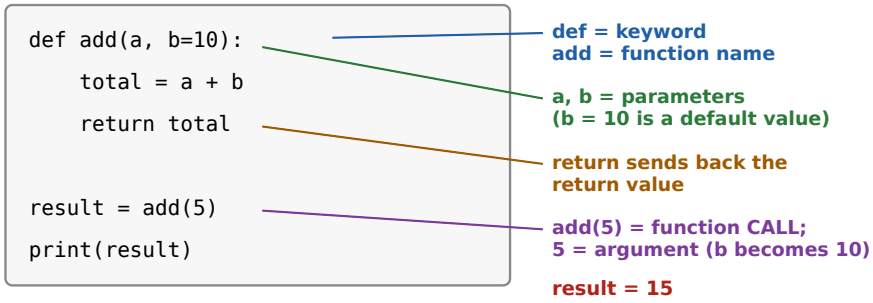
Quick: P25 v **C**; viii **C**. P24 xi **D**; xiii **B**; b(i) **list**, (iv) **dictionary**; c(ii) **TRUE**.

7 · FUNCTIONS & RECURSION

P25 Q1a(vii),b(ii),Q5b–e · P24 Q1a(xiv),b(v),Q2c,Q6c

Functions

Anatomy of a function



Local variables (total) exist only inside the function.
If a function is defined twice with the same name, only the LAST definition is kept.

Anatomy of a function: keyword, name, parameters, return value, call and arguments.

- A **function** is a **reusable block of code that performs a specific task** and can be called many times **P25 Q1b(ii): function** **P24 Q1b(v): function**.
- **Parameter** = the variable in the definition (`def hello(f_name)`); **argument** = the value passed when calling (`hello("Sonam")`).
- `return` sends a value back to the caller; `print` only displays.
- **Default argument:** `def add(a, b=10)` — if the caller leaves out `b`, it is 10. `add(5)` → 15 is valid; `add()` fails (a is required); passing 3 arguments fails (only 2 parameters) → **D** **P25 Q1a(vii)**.
- **Local variable:** created inside a function and usable only there; **global variable:** created outside.

```
def add(a, b=10):
    return a + b

print(add(5))        # 15
print(add(5, 20))    # 25
```

Reading code (traces)

```
def mystery_function(a):
    return a*2 + 3
result = mystery_function(5)
print(result)          # 5*2 + 3 = 13

def addition(x, y):    # 1 = parameters (x, y)
    add = x + y
    return add
cal = addition(10, 5)  # 2 = function call, arguments 10 and 5
print(f'Addition:{cal}')   # Addition:15
```

Recursion

- A **recursive function calls itself** on a smaller problem until it reaches a **base case** that stops the calls. It needs (1) a base case and (2) a recursive case that moves toward it.

countConsonants("Bhutan") unfolds step by step

"Bhutan"	B is a consonant	**1 + count("hutan")**
"hutan"	h is a consonant	**1 + count("utan")**
"utan"	u is a vowel	**count("tan")**
"tan"	t is a consonant	**1 + count("an")**
"an"	a is a vowel	**count("n")**
"n"	n is a consonant	**1 + count("")**
""	BASE CASE	**return 0**

Going back up (unwinding):

count("") = 0

count("n") = 1 + 0 = 1

count("an") = 1

count("tan") = 1 + 1 = 2

count("utan")= 2

count("hutan")= 1 + 2 = 3

count("Bhutan")= 1 + 3 = 4

Answer: 4 (B, h, t, n)

The recursion for countConsonants("Bhutan").

```
def countConsonants(text):
    if text == "":                      # base case (paper shows " ")
        return 0
    elif text[0].isalpha() and text[0].lower() not in "aeiou":
        return 1 + countConsonants(text[1:])
    else:
        return countConsonants(text[1:])

print(countConsonants("Bhutan"))        # 4
```

Solved questions

P25 Q5b [2] Parameter: **f_name** (in `def hello(f_name)`). Local variable: **message** (created inside the function). (`first_name` is a global variable outside the function.)

P25 Q5c [1] Python does not support two functions with the same name: the **second definition replaces the first**. So only `greet(name)` exists; `greet("Tashi")` prints "Hello Tashi", but calling `greet()` gives a **TypeError** (missing argument).

P25 Q5d [2]

```
def square(n=5):          # default argument
    return n * n

print(square(4))          # called with a number -> 16
print(square())           # called without -> uses 5 -> 25
```

P25 Q5e [4] (i) **Base case:** when the text is empty (no characters left) the function returns **0** and stops the recursion. **(check)** the paper prints `" "`; with a space the empty string would not match and the function would fail. (ii) **Consonant check:** `text[0]` is the first letter; it must be a letter (`isalpha()`) **and** its lowercase must **not** be in "aeiou". (iii) **Recursion:** if the first letter is a consonant it returns **1 + countConsonants(rest of the text)** (`text[1:]`), otherwise it returns the count of the rest without adding 1; each call handles one letter and passes the shorter text on until the base case; the 1s are added when the calls return. (iv) "Bhutan" has **4 consonants** (B, h, t, n); u and a are vowels.

P24 Q2c [4] (i) The call `add_numbers(5)` gives only **one argument** but the function needs two (**TypeError**). Corrected:

```
def add_numbers(a, b):
    return a + b
print(add_numbers(5, 3))     # 8   (or give b a default value: def
add_numbers(a, b=0))
```

(ii) **13**: `mystery_function(5)` returns 5 × 2 + 3 = 13, which is stored in `result` and printed.

P24 Q6c [3] (i) **1 = parameters** (x, y) of the function definition; **2 = function call** (with the arguments 10 and 5) **(check labels in your paper)**. (ii) Keyword: **def** (and **return**); function name: **addition**; return value: **add (= 15)**. Output: **Addition:15**.

P24 Q1a(xiv) / P25 Q1a(vii) Profit function with `sp - cp` → **D** (profit 20). `add(5)` with `b=10` default → valid call **D**.

Quick: P25 vii **D**; b(ii) **function**. P24 xiv **D**; b(v) **function**.

8 · ALGORITHMS & FLOWCHARTS

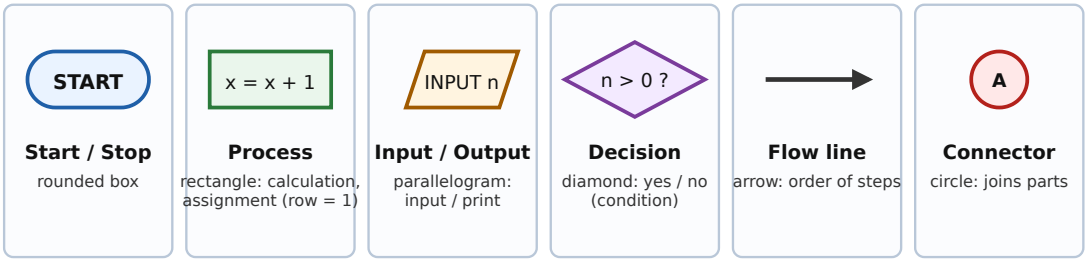
P25 Q1a(xii),c(v),Q3d,Q6e · P24 Q6a

Algorithm vs flowchart

- An **algorithm** is a **step-by-step set of instructions to solve a problem** (written in words or pseudocode) P25 Q1a(xii): D. A **flowchart** is a **diagram of the algorithm** using standard symbols and arrows.

	Algorithm	Flowchart
Form	written **steps** in words / pseudocode	**diagram** with symbols and arrows
Easy to	write quickly	see the flow and decisions at a glance
Uses	plan the logic before coding	explain the logic visually, find errors in the flow

Flowchart symbols



Process = rectangle; Input/Output = parallelogram (P25 Q1c(v) is FALSE)

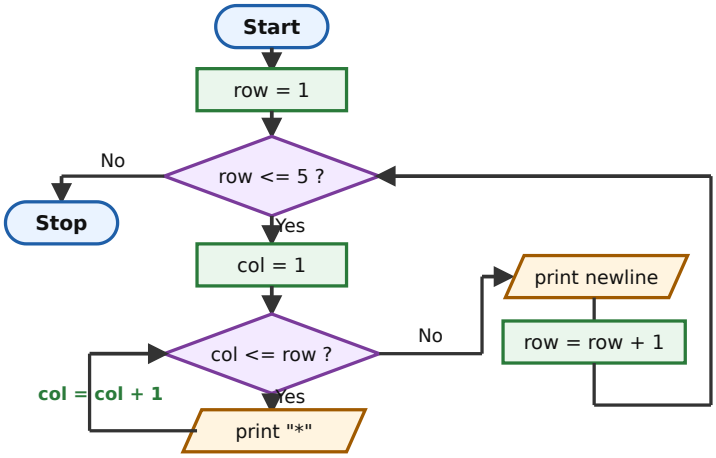
Standard flowchart symbols.

- **Process** is a **rectangle**, not a parallelogram; the parallelogram is for input/output. “In a flowchart, process is represented by a parallelogram” is **FALSE** P25 Q1c(v).

Flowchart to code

- Convert each symbol: **input** → `input()`; **process** → assignment or calculation; **decision** → `if` (or `while` when the arrow goes back); **print** → `print()`; **start/stop** → begin/end.

Flowchart for the star triangle (P25 Q6e)



Python version

```
row = 1
while row <= 5:
    col = 1
    while col <= row:
        print("*", end="")
        col += 1
    print()
    row += 1
```

The 2025 flowchart (Q6e) and its Python program.

Solved questions

P25 Q3d [2] Algorithm vs flowchart (any two differences): (1) an algorithm is a **written list of steps**, a flowchart is a **diagram**; (2) an algorithm uses **words or pseudocode**, a flowchart uses **symbols and arrows**; (3) a flowchart shows **decisions and loops visually**, which is easier for beginners; an algorithm is faster to write.

P25 Q6e [4] The flowchart sets row = 1; **while row <= 5**: set col = 1, **while col <= row** print “*” (same line) and add 1 to col; then print a newline and add 1 to row. Program (right-angled triangle of stars):

```
row = 1
while row <= 5:
    col = 1
    while col <= row:
        print("*", end="")
        col += 1
    print()          # newline
    row += 1
```

Output:

```
*
**
***
****
*****
```

P24 Q6a [4] Smaller of two numbers (flowchart: input num1, input num2, decision `num1 < num2`: yes → print num1, no → print num2):

```
num1 = int(input("Enter value of num1: "))
num2 = int(input("Enter value of num2: "))
if num1 < num2:
    print("Smallest is", num1)
else:
    print("Smallest is", num2)
```

Quick: P25 xii **D**; c(v) **FALSE**.

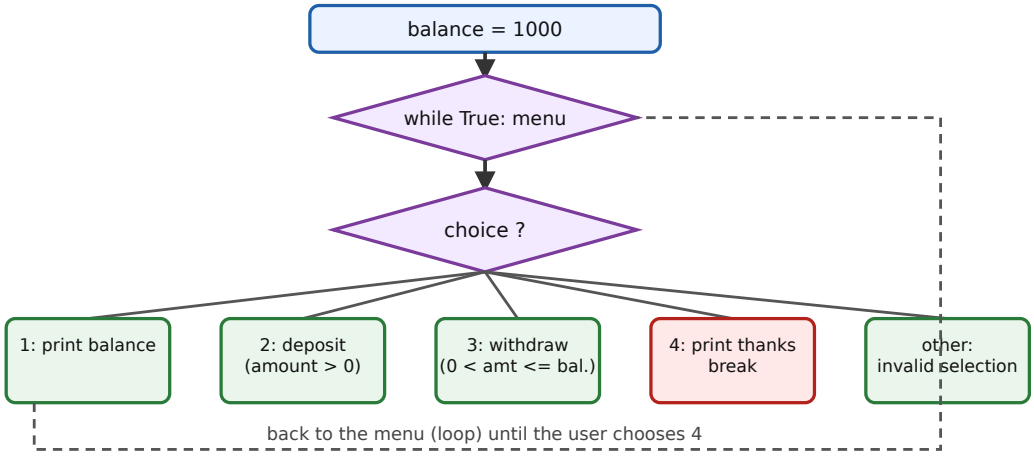
9 · LONG PROGRAMS (Q7 EVERY YEAR)

P25 Q7 mobile banking · P24 Q7 school mess system

How to score full marks in Q7

- Read the numbered steps (i)–(vii) and write **one block of code for each**, in order. Each step is worth the marks shown.
- Use `while True:` for a **menu that repeats until the user exits** and `break` to leave it. Use `if / elif / else` for the choices and **validate** inputs (amount > 0, amount ≤ balance).
- Convert input with `int()` or `float()`. Keep the **indentation** correct.

Mobile banking program: logic



Logic of the mobile banking program.

2025 Q7: Mobile banking simulation [10]

```
balance = 1000                                  # (i) [1]

while True:                                     # (ii) [2] menu
repeats
    print("===== Menu =====")
    print("1. Check Balance")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Exit")
    choice = int(input("Enter your choice (1-4): "))

    if choice == 1:                             # (iii) [1]
        print("Current balance: Nu.", balance)

    elif choice == 2:                           # (iv) [2]
        amount = float(input("Enter amount to deposit: "))
        if amount > 0:
            balance += amount
            print("New balance: Nu.", balance)
        else:
            print("Error: amount must be greater than zero.")

    elif choice == 3:                           # (v) [2]
        amount = float(input("Enter amount to withdraw: "))
        if amount > 0 and amount <= balance:
            balance -= amount
            print("Updated balance: Nu.", balance)
        else:
            print("Error: invalid amount or insufficient balance.")

    elif choice == 4:                           # (vi) [1]
        print("Thank you for banking with us!")
        break

    else:                                       # (vii) [1]
        print("Invalid selection. Please choose a valid option.")
```

- **Marking points:** balance variable (1); loop + menu + input (2); choice 1 (1); deposit with validation and update (2); withdraw with both conditions and update (2); exit message and break (1); invalid choice (1).

2024 Q7: School mess management [10]

```
def mess_system():
    print("School Mess Management Program")            # (i)

    items = ["rice", "pulses", "meat", "potato"]
    mess_qty = []                                       # (iii) list
    for item in items:                                  # (ii) ask 4
quantities
        qty = float(input("Enter quantity of " + item + " (kg): "))
        mess_qty.append(qty)

    if mess_qty[0] >= 1200:                             # (iv) rice
check
        print("You have enough rice")
    else:
        mess_qty[0] = float(input("Enter new quantity of rice: "))
        print(mess_qty)                                 # updated list

    cost = [40, 50, 250, 30]                            # (v) cost per
kg
    total = 0
    for i in range(4):
        total = total + mess_qty[i] * cost[i]
    print("Total expenditure: Nu.", total)

    if total <= 50000:                                  # (vi) budget
        print("You have enough budget")
    else:
        print("You don't have enough budget")

mess_system()                                           # call the
function
```

- Total expenditure = quantity × cost for each item, added together. Example: rice 1300, pulses 200, meat 100, potato 500 → 52 000 + 10 000 + 25 000 + 15 000 = **Nu. 102 000** → "You don't have enough budget".

Common mistakes to avoid

- Missing **colon** after `if`, `while`, `def`; wrong **indentation**; using `=` instead of `==` in a condition.
- Forgetting `int()` / `float()` around `input()`; forgetting to **update** the balance; forgetting `break` for the Exit option; forgetting to **call** the function.
- Case: `True`, `False` start with capitals; `pass`, `continue`, `break` are lowercase.

Practice: without looking, rewrite (a) the banking menu, (b) the mess system, (c) the star triangle, (d) a function with a default argument, (e) sum 1–50 with `for` and `while`.

SECTION A ANSWER KEY — 2025 & 2024

each MCQ is worth 2 marks · (check) = faulty or reasoned answer

2025 — Q1(a) MCQ [15 × 2]

No.	Ans	Why (short)
i	**D**	use of the song must match its licence
ii	**C**	loop prints 1, 2, 3 then breaks at i == 3
iii	**B**	Google Docs: simultaneous editing
iv	**C**	APA 7: sentence-case italic title, initial T.
v	**C**	lists change, tuples stay fixed
vi	**A**	private cloud: control and security
vii	**D**	add(5) uses default b = 10
viii	**C**	data[1:4] = [20, 30, 40]
ix	**D**	line chart shows a trend over months
x	**A**	VLOOKUP("Tashi Dendup", B2:C5, 2, FALSE)
xi	**C**	Python strings are case sensitive: False
xii	**D**	algorithm = step-by-step instructions
xiii	**B**	BY-NC: remix, not for profit
xiv	**D**	number of books is an int
xv	(check)	faulty item: AVERAGEIF would give 75; no option matches

2025 — (b) blanks and (c) true/false [5 + 5]

	(b)	(c)
i	**servers**	**TRUE**
ii	**function**	**FALSE** (RANK gives position, not alphabetical)
iii	**in-text**	**TRUE**
iv	**continue**	**TRUE** (with the author's consent)
v	**NOT**	**FALSE** (process = rectangle)

2024 — Q1(a) MCQ [15 × 2]

No.	Ans	Why (short)
i	**C**	Data Validation (whole numbers 1–100)
ii	**D**	PaaS: develop and run apps
iii	**B**	=C2*D2 (units sold × unit price)
iv	**A**	range(1,15) with i % 2 == 0 prints 2–14
v	**B**	APA 7 book: sentence-case italic title, no city
vi	**B**	NOT reverses a logical value
vii	**D**	for loop for a known number of times
viii	**D**	Google Slides for a shared presentation
ix	**D**	6 sides, 60° turns: hexagon
x	**B**	CC lets others use under conditions
xi	**D**	pop(2) removes "inbuilt"
xii	**B**	school_1 is a valid name
xiii	**B**	my_tuple[1] = 20
xiv	**D**	profit = sp − cp
xv	**D**	=SUMIF(B2:B6,"IT",C2:C6)

2024 — (b) blanks and (c) true/false [5 + 5]

	(b)	(c)
i	**list**	**TRUE**
ii	**plagiarism**	**TRUE** (set)
iii	**copyright**	**FALSE** (vague names are bad)
iv	**dictionary**	**TRUE**
v	**function**	**FALSE** (check: creators keep copyright)

LAST-DAY CHECKLIST

tick when you can write or explain it without looking

- ☐ **Cloud:** IaaS / PaaS / SaaS (who manages what); private, public, hybrid; two benefits (no upfront cost, scalability, backup); Google Docs, Slides, Forms, Drive.
- ☐ **Copyright & citation:** copyright vs Creative Commons; BY, NC, SA, ND; plagiarism; APA book format (sentence case, italic title, no city); in-text vs reference list; legal use of images.
- ☐ **Excel:** MAX, AVERAGE(B2:E2), SUMIF(range, criteria, sum range), VLOOKUP (first column, col number, FALSE), NOT, RANK, data validation, which chart for which data.
- ☐ **Python basics:** valid names, case sensitivity, int / float / str / bool, == vs =, and / or / not, Boolean table, if / elif / else.
- ☐ **Loops:** range(start, stop, step) (stop excluded), for vs while, break / continue / pass, tracing output, sum 1–50, Turtle polygon turns.
- ☐ **Data structures:** list [ ], tuple ( ), set { }, dictionary {k: v}; immutability of strings and tuples; indexing, slicing, negative index; pop; set difference; add a key to a dictionary.
- ☐ **Functions:** def, parameter vs argument, return, default argument, local variable, two functions with the same name, recursion (base case + recursive case).
- ☐ **Flowcharts:** symbols (rectangle process, parallelogram I/O, diamond decision); algorithm vs flowchart; flowchart to code (star triangle, smaller of two numbers).
- ☐ **Long programs:** menu loop with while True and break; validation (amount > 0 and ≤ balance); list of quantities; total = quantity × cost; call the function.
- ☐ **In answers:** indentation and colons; write the exact output; explain with “because…”; count the marks and give that many points.